Deadlock Intro:

Dr. Jit Mukherjee

Introduction

- Multiprogramming - several process compete for several resource
  - A resource not available, requesting process (A) -> waiting state or busy waiting
  - The resource held by another process, which is waiting for a resource from A.
- Resource and instances
  - Identical and nonidentical instances. Physical and lopgical resource
- A process must request before using and release after using
  - Number of requested resource < total number of resource available
- Normal mode of operations
  - Request - if not granted the process must wait, request(), open(), allocate()
  - Use - operate on resource.
  - Release - release the resource. release(), close(), free()
- A set of processes is in a deadlock state when every process in a set is waiting for an event that can be caused only by another process in the set

Deadlock Characterizations

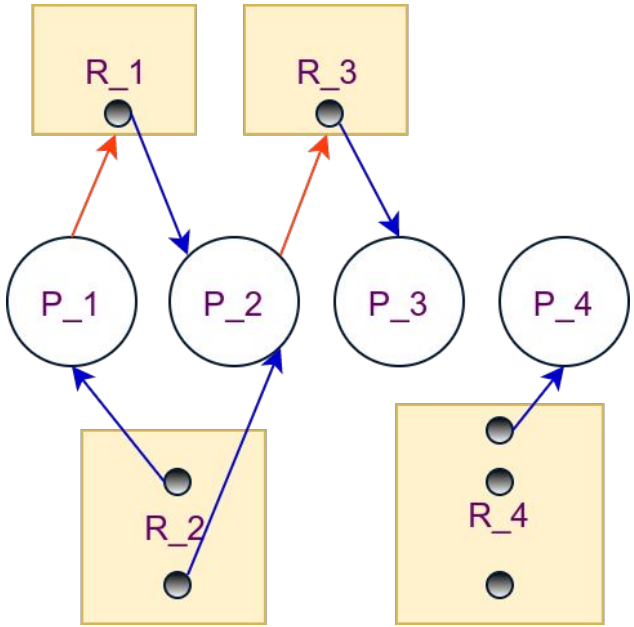
- Process never finishes execution, system resource tied up, prevent others
- Necessary Conditions
  a. **Mutual Exclusion** - At least one resource -> nonsharable mode. Only one process can use the resource at a time. Other waits
  b. **Hold and Wait** - A process must be holding at least one resource and waiting to acquire additional resources which is being held by other processes
  c. **No Preemption** - Resources can not be preempted. Process leaves resource after completing.
  d. **Circular wait** - A set {P_0, P_1, .., P_n} must exists such that
     - P_0 is waiting for a resource held by P_1
     - P_1 is waiting for a resource held by P_2
     - P_2 is waiting for a resource held by P_3
     - ……
     - P_n is waiting for a resource held by P_0

Resource Allocation Graph

- Set of vertices and edge. Vertices ->
  - P = {P_0, P_1, .., P_n} -> Active process -> Circles Generally
  - R = {R_0, R_1, .., R_m} -> Available resources -> Rectangle Generally -> Dots are instances
- Edge -> Directed edge
  - P_i -> R_j means P_i has requested an instance of resource type R_j but waiting right now
    - **Request Edge**
  - R_j -> P_i means an instance of R_j is allocated to P_i
    - **Assignment Edge**
- P_i requests R_j, a request edge is inserted. When given it is changed to assignment edge.

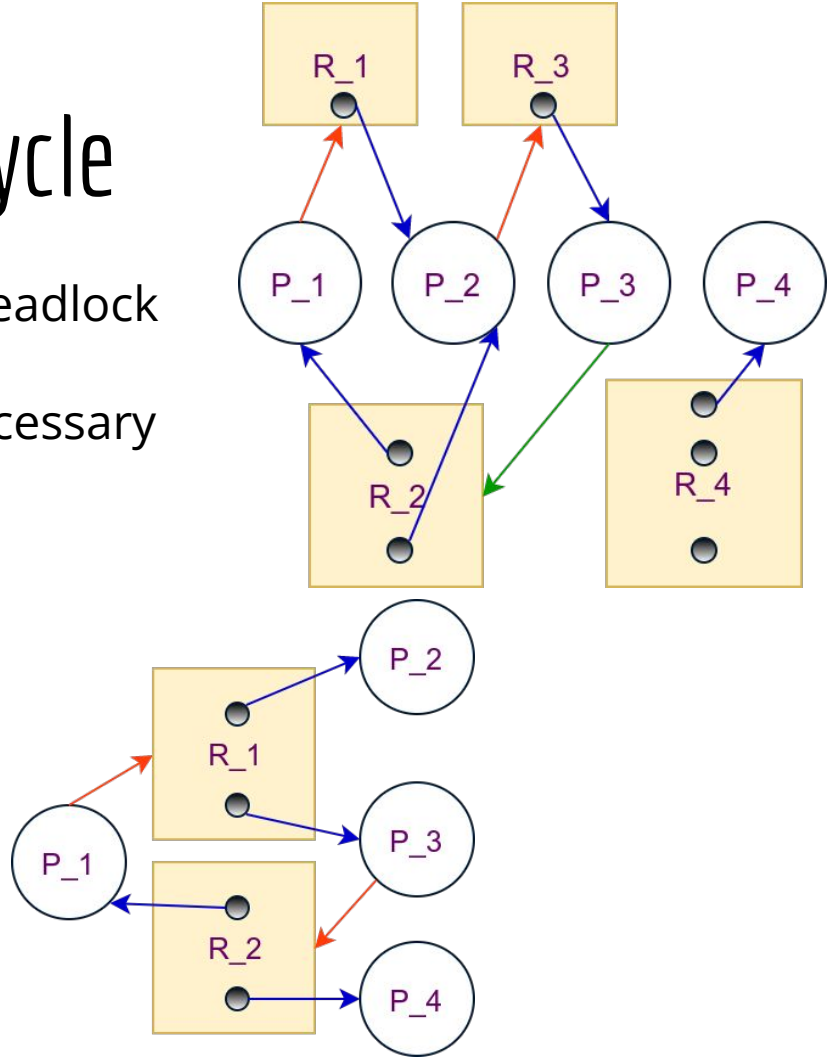
Resource Allocation Graph

- The sets P, R, and E
  - P = {P_1, P_2, P_3, P_4}
  - R = {R_1, R_2, R_3, R_4}
  - E = { P_1 -> R_1, P_2 -> R_3, R_1 -> P_2, R_2 -> P_1, R_2 -> P_2, R_3 -> P_3, R_4 -> P_4}
- Resource instances
  - R_1 -> 1, R_2 ->2, R_3 -> 1, R_4 ->3
- Process States
  - P_1 is holding an instance of R_2 and waiting for R_1
  - P_2 is holding an instance on both R_2, and R_1 and waiting for R_3
  - P_3 holding an instance of R_3
  - P_4 holding an instance of R_4
- No cycle -> no deadlock. If cycle -> may be.



Resource Allocation Graph: Cycle

- Resources -> Once instance. If cycle -> deadlock
  - Cycle here necessary and sufficient condition
- Resources -> multiple instance. Cycle necessary but not sufficient condition.
- Two cycles
  - P_1 -> R_1 -> P_2 -> R_3 -> P_3 -> R_2 -> P_1
  - P2 -> R_3 -> P_3 -> R_2 -> P_2
  - P_1, P_2, and P_3 are deadlocked.
- P_1 -> R_1 -> P_3 -> R_2 -> P_1



Deadlock Prevention

- One of these three way for handling deadlock
  - Protocol to prevent or avoid deadlock
  - Enter a deadlock, detect and recover
  - Ignore the problem. Windows, Linux. Application developer
- At least one of the four necessary conditions never holds
  - Deny Mutual Exclusion - No problem with sharable resources, ex. readonly file. Concurrent access can be granted. Practically impossible as many resources are non-sharable.
  - Deny Hold and Wait -
    - Whenever a process request resource it does not hold any.
    - Request all the resources at the beginning.
    - Allow request resource when it has none. Release resource before having another
    - Disadvantage ->
      - Resource allocation is poor. Unused for long time
      - Starvation. Needs several resources.

Deadlock Prevention

- Deny No Preemption
  - If a process is holding some resources and request other which can not be allocated -> All resources currently being hold must be preempted / released.
    - The process will restart when it can gain all the old and new ones.
  - A process (X) request resource -> check availability
    - If yes, assign them. No, check where they are allocated, other process
    - That other process waiting for additional resources. Assign to X
    - Resource not available, not being held. X waits and X's resources may be preempted
      - Restarted -> allocated new resources and recovers old ones
- Deny Circular Wait - Sort resources -> number of instances F(tape drive) = 1, F(dist drive) = 5, F(printer) = 12. Process request resource increasing order
  - Process request or has R_i, can request R_j if F(R_j) > F(R_i).
  - A process request R_j, Released R_i F(R_i) >= F(R_j)
  - {P_0, P_1,.. P_n} -> [R_i -> P_i] + [P_(i+1) -> R_i] -> F(R_i) < F(R_(i+1)). Failed for R_n and R_0
  - Responsibility of Application developer. Normal order of usage may defy.
- *Witness* - Software uses mutual-exclusion locks to prevent deadlock